

2-2 · CTE, Window Function 분석 쿼리 · 이론 1H

서브쿼리의 한계와 CTE·Window Function 개념

밑준비에 이름을 붙이고, 행을 접지 않고 집계한다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

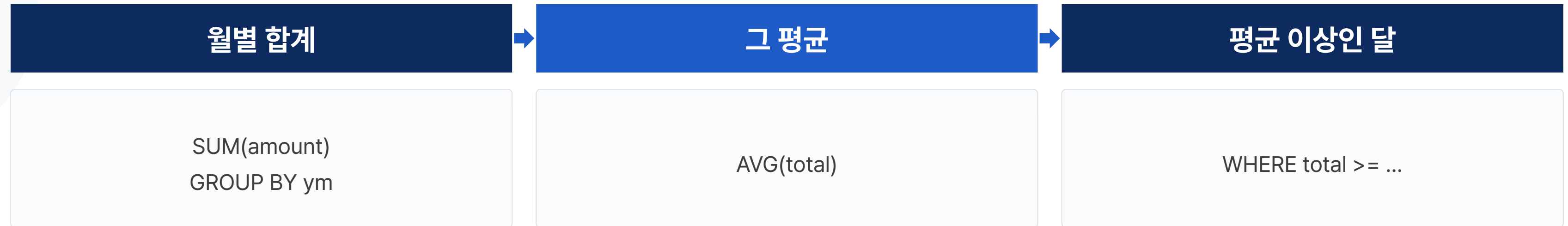
1시간 뒤에는

- ① 중첩 서브쿼리가 왜 읽기 어려운지 말할 수 있다
- ② CTE(WITH 절)로 중간 결과에 이름을 붙여 위에서 아래로 읽게 만든다
- ③ GROUP BY(행을 접음)와 Window Function(행 유지)을 구별한다
- ④ OVER 안의 PARTITION BY·ORDER BY 가 무엇을 정하는지 설명한다

다음 1H 실습에서 서브쿼리를 CTE로 바꾸고, 1.5H 실습에서 순위·누적·전월 비교를 직접 돌립니다

1. 서브쿼리의 한계

분석 질문은 대개 여러 단계다



서브쿼리 중첩 = 안쪽 SELECT 부터 읽어야 해서 눈이 계속 되돌아간다

서브쿼리로 쓰면 안쪽부터 거꾸로 읽어야 하고, 같은 집계를 두 번 적게 된다 — 한쪽만 고치면 조용히 틀린다

2. CTE

같은 결과 — 서브쿼리 중첩과 CTE

서브쿼리 중첩

```
FROM (SELECT ...) t WHERE total >= (SELECT AVG ...  
FROM (SELECT ...) t2)
```

- 안쪽부터 거꾸로 읽는다
- 같은 GROUP BY 를 두 번 적는다
- 고칠 때 두 곳을 같이 고쳐야 한다

CTE (WITH 절)

```
WITH monthly_total AS (...) SELECT ... FROM  
monthly_total
```

VS

- 위에서 아래로 읽힌다
- 집계를 한 번만 적는다
- 고칠 때 한 곳만 고친다

2. CTE

CTE — 밑준비에 이름을 붙인다

1 WITH 이름 AS (쿼리)

이름 붙은 임시 결과 — 같은 쿼리 안에서 테이블처럼 참조한다

2 실제 테이블이 아니다

그 쿼리가 도는 동안만 존재한다

3 여러 개는 쉽표로

WITH a AS (...), b AS (SELECT ... FROM a) — 뒤의 CTE 가 앞의 CTE 를 참조할 수 있다

4 단계를 차례로 쌓는다

밑준비 → 조리 → 플레이팅처럼 긴 쿼리를 나눈다

CTE 는 MySQL 8.0 부터 — 5.7 이하는 WITH 자체가 문법 오류. `SELECT VERSION();` 으로 확인

3. Window Function

GROUP BY 는 행을 접고, Window 는 옆에 붙인다

GROUP BY

원래 행이 사라진다 — 합계를 구하면 개별 월이 없어진다

- 반 전체를 「반 평균 하나」로 접는다
- 행 수가 줄어든다
- 「이 달이 전체의 몇 %」를 한 번에 못 구한다

VS

Window Function

행을 접지 않고 집계한다 — 각 행 옆에 집계값을 나란히 적는다

- 학생 성적표 옆에 반 평균을 적어 준다
- 결과 행 수 = 입력 행 수 (결정적 차이)
- OVER 절이 집계할 창을 범위를 정한다

3. Window Function

OVER 안의 ORDER BY 가 창을 좁힌다 — 가장 헷갈리는 지점

ORDER BY 없음

SUM(amount) OVER (PARTITION BY category)

- 창 = 파티션 전체
- 모든 행에 같은 값(전체 합계)
- PARTITION BY 도 생략하면 전체가 한 그룹

VS

ORDER BY 있음

SUM(amount) OVER (... ORDER BY ym)

- 창 = 파티션 처음부터 현재 행까지
- 그래서 누적이 된다
- 기본 창은 RANGE — ORDER BY 값이 같은 행(동점)도 함께

3. Window Function

자주 쓰는 Window 함수

함수	하는 일	결과의 특징
ROW_NUMBER()	순번을 매긴다	동점이어도 1, 2, 3
RANK()	등수를 매긴다	1, 1, 그다음 3 (건너뛴)
DENSE_RANK()	등수를 매긴다	1, 1, 그다음 2 (안 건너뛴)
SUM/AVG/COUNT OVER	누적·이동 집계	—
LAG(컬럼, n)	n칸 앞 행의 값	없으면 NULL
LEAD(컬럼, n)	n칸 뒤 행의 값	없으면 NULL
FIRST_VALUE / LAST_VALUE	창의 첫 값 / 마지막 값	—

「상위 N개」는 ROW_NUMBER — RANK 는 동점 때문에 N개보다 많이 나올 수 있다. 공동 1위를 모두 보려면 RANK

4. 계산 순서

우리가 쓰는 순서와 DB 가 계산하는 순서는 다르다

순서	단계	이 시점에 쓸 수 있는 것
1	FROM · JOIN	원본 테이블의 컬럼
2	WHERE	원본 컬럼 (집계·Window 는 아직 없음)
3	GROUP BY	—
4	HAVING	집계 함수 결과
5	SELECT (Window 계산)	여기서 비로소 순위·누적
6	ORDER BY	SELECT 의 별칭도 가능
7	LIMIT	—

Window 는 5번, WHERE 는 2번 — WHERE rn = 1 은 「그런 컬럼 없다」. 해결은 CTE 로 한 겹 감싸고 바깥에서 거르기

이 1시간에서 꼭 가져갈 것

- 1 CTE 는 중간 결과에 이름을 붙여 쿼리를 위에서 아래로 읽히게 만든다
- 2 GROUP BY 는 행을 접고, Window Function 은 행을 그대로 두고 옆에 붙인다
- 3 OVER 안의 PARTITION BY 는 그룹, ORDER BY 는 창의 범위를 정한다
- 4 ORDER BY 가 있으면 누적, 없으면 파티션 전체 합계다
- 5 Window 결과는 WHERE 에서 못 쓴다 — CTE 로 감싸고 바깥에서 거른다

2-2 · 이론 1H 마무리

다음은 CTE 실습 (1H)

분석용 월별 매출 테이블을 만들고, 서브쿼리로 쓴 것을 CTE 로 바꿔 가며 차이를 몸으로 익힙니다

✓ monthly_sales 테이블 준비 (9행)

✓ 중복 INSERT 를 일부러 넣어 UNIQUE 확인

✓ 서브쿼리 → CTE 로 바꾸기

✓ 다중 CTE 체이닝 — 비중(%) 계산

준비할 것

start → port: 3306 확인 → LANG 붙여 접속